

Functional and Non-Functional Requirements – DarshanEase: Smart Temple Darshan Ticket Booking System

Project Report: Requirement Analysis & Specification

This report consolidates the functional, non-functional, and dependency requirements required to design and implement the DarshanEase Smart Temple Darshan Ticket Booking System in a structured, verifiable, and implementation-ready format.

Introduction

Requirement analysis is a foundational activity in software engineering that determines *what* a system must do before defining *how* it will be built.

For a public-facing platform such as DarshanEase, incomplete or ambiguous requirements can lead to security risks, operational inefficiencies, poor user experience, and inability to meet temple capacity constraints during peak periods.

A rigorous requirement analysis process improves clarity among stakeholders by translating the expectations of devotees, temple organizers, and administrators into precise, testable statements.

It also establishes measurable acceptance criteria, supports accurate effort estimation, reduces rework during development, and ensures that key constraints such as reliability, scalability, and accessibility are treated as first-class objectives.

Accordingly, this document formalizes the functional and non-functional requirements for the DarshanEase Smart Temple Darshan Ticket Booking System and provides structured artifacts (tables, dependencies, and priorities) to guide implementation and validation.

1. Functional Requirements

Functional requirements define the observable services and behaviors that DarshanEase must provide to end users and administrators. These requirements focus on the core

workflows of user onboarding, secure authentication, darshan slot discovery, booking, payment handling (if applicable), ticket generation, and entry verification. They also include administrative capabilities for schedule configuration, capacity control, booking oversight, and operational reporting. The following table specifies detailed functional requirements along with their expected behavior to enable clear development and testing.

FR ID	Requirement	Description (Detailed)	Primary Stakeholder
FR-01	User Registration	The system shall allow devotees to register using name, mobile/email, and password (or OTP-based registration) with validation to prevent duplicate accounts.	Devotee
FR-02	Secure Login & Session	The system shall authenticate users and maintain secure sessions, enabling logout and session expiry to protect accounts on shared devices.	Devotee, Admin
FR-03	Temple & Darshan Slot Browsing	The system shall display available temples and darshan time-slots with real-time remaining capacity, date selection, and slot details (duration, rules, instructions).	Devotee
FR-04	Online Booking	The system shall allow devotees to select a date and slot, enter required devotee details, confirm the booking, and receive a booking reference number.	Devotee
FR-05	Payment Handling (If Enabled)	The system shall support online	Devotee, Temple Organizer

		payment integration where applicable, record payment status, and ensure bookings are confirmed only after successful payment.	
--	--	---	--

FR ID	Requirement	Description (Detailed)	Primary Stakeholder
FR-06	QR-Code Ticket Generation	The system shall generate a unique QR-code ticket upon confirmation, embedding booking ID and verification metadata to support fast entry validation at the temple gate.	Devotee, Temple Staff
FR-07	Ticket Download & Notification	The system shall enable ticket download/print and send booking confirmation via SMS/email/within-app notification containing slot details and instructions.	Devotee
FR-08	Booking History & Cancellation	The system shall provide a booking history view and support cancellation/rescheduling as per temple rules, updating capacity and issuing refund status where applicable.	Devotee, Admin
FR-09	Slot Capacity Allocation	The system shall allocate bookings against slot capacity in real time, prevent overbooking, and lock a seat during checkout to avoid race conditions.	Temple Organizer, Admin

FR-10	Admin Dashboard & Management	The system shall provide administrators with tools to create/update slots, set capacity, view bookings, manage users, and access reports/analytics for operational decisions.	Admin
-------	------------------------------	---	-------

2. Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and operational standards that DarshanEase must satisfy to ensure trust, reliability, and usability in real-world temple environments. Given the sensitive nature of personal data, high peak loads during festivals, and the need for rapid verification at entry points, the system must be designed for strong security, high availability, responsive performance, and consistent user experience across devices. The following table summarizes key non-functional requirements that guide architecture, implementation, and validation.

NFR ID	Quality Attribute	Requirement	Measurement / Criterion
NFR-01	Security	All sensitive data shall be protected via secure authentication, encrypted transport, and role-based access control for administrative operations.	HTTPS enforced; hashed passwords; RBAC; audit logs for admin actions.
NFR-02	Performance	The system shall respond quickly to slot search and booking operations to avoid user drop-off during peak traffic.	Common actions complete within acceptable response times under expected load.
NFR-03	Availability & Reliability	The system shall remain operational during high-demand periods and recover gracefully from failures.	High uptime target; automated backups; graceful error handling; retry mechanisms.
NFR-04	Usability & Accessibility	The user interface shall be simple, multilingual-ready, and accessible across desktop, tablet, and mobile devices.	Responsive UI; readable typography; clear navigation; accessible form interactions.
NFR-05	Scalability	The architecture shall support increasing devotees, multiple temples, and peak festival demand without major redesign.	Horizontal scaling supported at the application and database tiers.

3. Module Dependencies

Module dependencies describe how major system components interact to deliver end-to-end booking and verification workflows. Understanding these dependencies is essential for implementing modules in a correct sequence, identifying integration points, and preventing failures that may arise from missing upstream services (e.g., booking confirmation without capacity validation). The dependency flow below reflects a typical devotee journey and the corresponding backend coordination required for secure booking and controlled entry.

- 1. User Interface (Web/Mobile) → Authentication Service → User Profile Management
 - 2. User Interface → Temple & Slot Catalog Service → Capacity/Inventory Manager
 - 3. Booking Service → Capacity/Inventory Manager → (Optional) Payment Gateway → Booking Confirmation
 - 4. Booking Confirmation → QR-Code Generator → Ticket Delivery (Download/Email/SMS)
 - 5. Entry Verification App/Portal → QR Scanner → Verification API → Booking Status Update (Checked-in/Used)
 - 6. Admin Dashboard → Slot & Capacity Configuration → Reporting/Analytics Module
-

4. Requirement Priority Matrix

The requirement priority matrix classifies requirements based on their importance to core operations and their urgency for initial release. Prioritization supports effective planning by ensuring that essential capabilities (such as secure booking, capacity control, and QR verification) are implemented first, while enhancements are scheduled for subsequent iterations. The categories below may be interpreted as: **High** (must-have for MVP), **Medium** (important but can follow), and **Low** (optional or future improvement).

Requirement	Type	Priority	Rationale
Online Booking (FR-04)	Functional	High	Central capability enabling devotees to reserve darshan slots digitally.
Slot Capacity Allocation (FR-09)	Functional	High	Prevents overbooking and ensures controlled crowd management.
QR-Code Ticket Generation (FR-06)	Functional	High	Enables fast verification and reduces entry time at temple premises.
Security Controls (NFR-01)	Non-Functional	High	Protects user data and prevents misuse of administrative features.
Admin Dashboard (FR-10)	Functional	Medium	Essential for ongoing operations, monitoring, and configuration.
Usability & Accessibility (NFR-04)	Non-Functional	Medium	Improves adoption for diverse user groups and device contexts.
Advanced Analytics (part of FR-10)	Functional	Low	Can be expanded post-MVP once stable booking and verification are achieved.

5. Expected Outcome

The functional and non-functional requirements documented in this report provide a complete and detailed blueprint for developing the DarshanEase Smart Temple Darshan Ticket Booking System.

These requirements have been carefully analyzed and structured to address the specific needs of devotees, temple organizers, and administrators, thereby ensuring stakeholder alignment and implementation clarity.

By digitizing the end-to-end darshan booking process, the system is expected to significantly improve temple visit management through secure online reservations and controlled access.

The proposed solution will eliminate long queues and reduce the dependency on manual ticketing processes that commonly lead to congestion, delays, and operational errors.

A key outcome is the QR-code ticket generation capability, which enables efficient verification, lowers entry time at temple premises, and supports reliable check-in tracking.

In addition, the slot allocation mechanism is designed to prevent overbooking and optimize temple capacity utilization, particularly during peak seasons and festival periods.

Centralized administration capabilities will empower temple management by providing real-time analytics, booking insights, and operational control for planning and crowd regulation.

The system's responsive user interfaces are intended to ensure accessibility across desktop, tablet, and mobile devices, enabling devotees to book darshan from any location with minimal friction.

From an architectural perspective, the scalable MERN (MongoDB, Express.js, React, Node.js) stack offers a robust foundation to accommodate future enhancements, multi-temple expansion, and increasing user loads.

Collectively, these requirements support a secure, reliable, and user-centric platform that aligns technological implementation with practical on-ground temple workflows.

Successful implementation will result in a modern, efficient, and user-friendly digital solution that transforms the traditional temple darshan booking experience while streamlining temple operations through technology.

Furthermore, the system represents a significant step toward digitizing religious tourism infrastructure in India, making temple visits more accessible and organized for millions of devotees.
